Individual Project: Waste Management

of a Company

Ali Ahammad (a81317@ualg.pt)

Course: DMI

Table of Contents

1. Introduction: 3
2. Project Outline 3
3. System Requirements 5
4. SQL 5
5. NoSQL: Dockerized Apache Cassandra 9
6. XML: Dockerized BaseX 13

1. Introduction:

To achieve both environmental sustainability and economic efficiency, effective waste management is essential. Businesses and governments can better oversee recycling, disposal, and waste collection processes by relying more and more on data-driven approaches. In this transition, centralised databases play a crucial role by offering reliable tools for monitoring trash generation patterns, guaranteeing legal compliance, and promoting well-informed decision-making. Organisations can improve operational transparency, cut down on inefficiencies, and adjust to changing environmental regulations by utilising these solutions.

This project's goal is to use a multi-database system that includes SQL, NoSQL, and XML technologies to create and construct an all-encompassing waste management framework. These databases include a variety of functions, such as handling customer contacts like billing and service messages, classifying items, maintaining waste disposal records, and evaluating compliance trends. The system may be modified to handle additional waste streams, such hazardous or electronic items, and is designed to serve a variety of sectors, including residential, commercial, and industrial. The theoretical underpinning of this research places a strong emphasis on sophisticated query strategies to glean insightful information. Accurate and effective data retrieval is ensured by methods like bag semantics, containment, and query unfolding. These techniques can, for example, pinpoint areas of noncompliance, draw attention to patterns in recyclable items, and maximise the use of resources in garbage collecting activities.

This system helps organisations find areas for improvement, streamline operations, and make strategic decisions by centralising and analysing waste management data. To meet changing demands, the multi-database strategy offers a flexible and scalable solution that supports sustainable waste management practices and long-term economic and environmental objectives. This project's usage of Docker made the deployment of database systems easier in Windows 11 environment and more which improved the consistency and efficiency of the implementation process.

2. Project Outline

I have This project focuses on querying waste management data from three types of databases as follows:

- **Structured Database**:

  XAMPP and phpMyAdmin are used to create a SQL database that supports a customer billing system. Following the "Pay-As-You-Throw" policy, it determines fees according to the kind and volume of garbage disposal.

- **Semi-Structured Database**:

  Customer contracts, which document agreements between the business and its clients, are stored in a NoSQL database that is implemented using Apache Cassandra (Dockerized).

- **Unstructured Database**:

Customer support and complaint handling are handled using an XML database built with BaseX (Dockerized). Additionally, membership access and associated correspondence are tracked by this database.

2.1 Policy Implementation and Customer-based Business Model

Many U.S.-based customer-centric waste management business models prioritize environmental compliance, efficiency, and transparency. These models often implement scheduled waste collection and dynamic billing systems, charging clients based on factors such as waste type, volume, and pickup frequency. Advanced tracking technologies ensure proper segregation of recyclables and adherence to eco-friendly disposal methods. The "Pay-As-You-Throw" policy incentivizes recycling and composting by linking charges to waste generation.

Greater Faro's current waste management system relies on conventional methods, with limited emphasis on dynamic billing or recycling incentives. Adopting aspects of the U.S. model, such as data-driven compliance tracking and dynamic billing systems, could enhance operational efficiency and sustainability. A centralized database system addressing regional challenges—like resource optimization and compliance gaps—would align Faro's waste management practices with international eco-friendly standards.

This project leverages the customer-based business model to design databases that incorporate waste management regulations, including landfill restrictions, recycling statutes, and waste segregation guidelines. Policies are encoded as rules, constraints, and queries within the databases, ensuring compliance and enabling automated decision-making. By integrating customer-focused practices with regulatory requirements, the system provides insights into waste generation trends and customer behaviour. For instance, the databases can:

Policy-Driven Database Framework:

- Embed policies as rules and constraints to ensure compliance with waste management guidelines.
- Handle waste categorization (e.g., hazardous, recyclable, biodegradable) based on regional regulations.

Customer centric Data:

- The database stores customer profiles, billing records, and waste disposal habits.

Operational benefits:

- Access to real-time data on collection, disposal, and recycling rates enhances service efficiency.
- Management can tailor programs, such as targeted recycling initiatives for residential areas or specialized waste disposal for industries.

The database system facilitates operational enhancements and strategic decision-making by fusing customer data with policy execution. Theories of query schema make it easier to conduct methodical searches across many data sources. Effective data extraction is made possible by strategies like query unfolding and conjunctive queries, which can be used to meet a variety of waste classification, tracking, and reporting needs.

3. System Requirements

3.1 Functional Requirements

a) Data Access:

- Use schema-based queries to access XML, SQL, and NoSQL data sources.
- Provide effective access to waste management-related data segments.

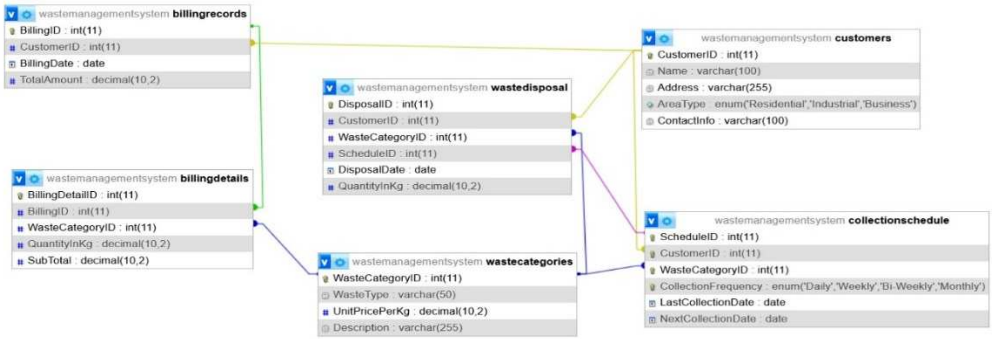
b) Schema Validation:

- To verify data consistency across sources, make sure integrity constraints and predicates are upheld.

4. SQL

4.1 Entity Relation Diagram

The data model has been designed to address the outlined problem domains, goals, and requirements by structuring data in a manner that is both flexible and comprehensive. It uses five primary tables:



4.2 Problem Domain and Constraints

Managing garbage of a large region effectively is a complex task that necessitates careful monitoring of waste production, collection, disposal, and billing. The goal is to develop a centralised database system that satisfies a range of client demands, including business, industrial, and residential ones, while abiding by environmental laws and preserving operational effectiveness.

To address the issues of trash disposal and client billing, this project uses a structured database that guarantees:

Precise Billing: charging customers according on the kind and quantity of rubbish they dispose of while following the "Pay-As-You-Throw" principle.

Collection Scheduling: Enabling dynamic waste collection scheduling to meet customer-specific requirements and fluctuating frequencies.

Data Integrity: Imposing restrictions to avoid data duplication and discrepancies, such as daily disposal limits (e.g., a maximum of 5000 kg per client) and specific timetables.

Regulatory Compliance: Aligning waste categorization and disposal with environmental guidelines while providing a transparent and auditable billing system.

4.3 Schemas Explanation and How They Align with the Constraints

Six primary tables make up the database and each of them is intended to handle a distinct waste management issue:

Clients_list:

- Records client data, such as name, address, and kind of area (business, residential, or industrial).
- Makes use of the `ClientID` primary key to guarantee each customer's unique identity.
- Helps in sector-based consumer classification for specific services.

WasteCategories:

- outlines the various categories of waste, including hazardous, recyclable, and biodegradable waste, as well as the corresponding costs per kilogramme.
- To ensure compliance with dynamic waste categorisation requirements, it incorporates a description column for clarity and future expansion.

CollectionSchedule:

- Customers are mapped according to their trash categories and frequency of pickup (e.g., *daily, weekly, monthly*).
- Using the `*unique_schedule*` constraint, it makes sure that every customer has a different set of schedule parameters, preventing duplicate entries for the same waste type and customer.

WasteDisposal:

- keeps track of actual garbage disposal actions, recording information such as the date, amount, and related schedules.
- enforces a daily cap of 5000 kg per customer to adhere to safety and operational limitations.
- Using foreign keys, there are direct links to customers and waste categories, guaranteeing accuracy and traceability.

BillingRecords:

- Stores overall billing data for each customer, including billing date and total amount.
- Facilitates a clear audit trail for financial transactions, ensuring transparency.

BillingDetails:

- Provides a granular breakdown of billing records, linking disposed quantities to specific waste categories and calculating subtotals.
- Automatically aligns with *UnitPricePerKg* from *WasteCategories* to calculate charges, maintaining accuracy in billing.

Scalability, consistency, and conformity to the operational and legal limitations specified in the issue domain are guaranteed by this model.

4.4 Relationships Between the Tables

The database structure makes use of a robust relational design, with carefully constructed associations to maintain data integrity and support efficient operations:

- **Clients_list → CollectionSchedule:**
  - a one-to-many arrangement in which every client can have several pickup schedules customised for various trash types and frequencies.
- **WasteCategories → CollectionSchedule and BillingDetails:**
  - Each waste category is appropriately priced and tracked thanks to a one-to-many linkage that links waste categories with schedules and billing information.
- **CollectionSchedule → WasteDisposal:**
  - A one-to-many relationship ensuring that each scheduled collection can result in multiple disposal entries, depending on customer activity.
- **Clients_list → BillingRecords:**
  - A one-to-many relationship that supports recurring billing cycles by allowing each client to have many billing records over time.
- **BillingRecords → BillingDetails:**
  - a one-to-many link that gives a thorough explanation of every account, matching the amounts disposed of with the costs and waste types.
- **Clients_list → WasteDisposal:**
  - A direct link that supports operational insights and customer responsibility by enabling tracking of all trash disposal actions for a specific customer.

These connections provide a smooth data flow across the system, enabling thorough tracking from waste generation to billing while respecting limitations such as disposal limitations and particular timetables. This approach offers a visible and scalable foundation for successfully tackling waste management's difficulties. and transparent framework for addressing the challenges of waste management effectively.

4.5 SQL Code

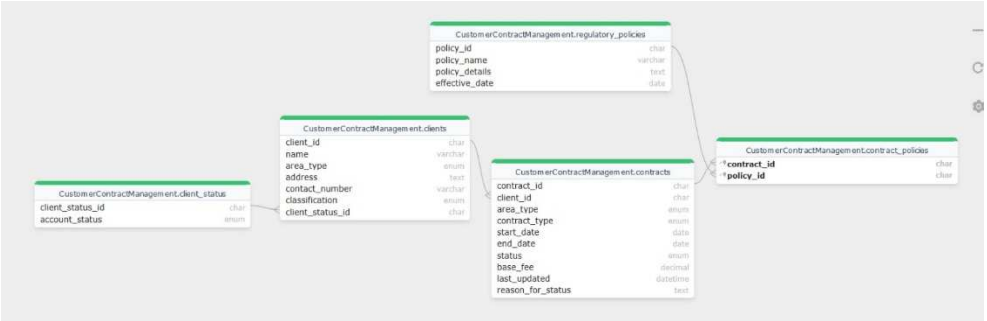
```sql
-- Create database
CREATE DATABASE WasteManagementSystem;
USE WasteManagementSystem;

-- Table: Customers
CREATE TABLE Clients_list (
    ClientID INT AUTO_INCREMENT PRIMARY KEY,
```

```sql
    Name VARCHAR(100) NOT NULL,
    fullAddress VARCHAR(255) NOT NULL,
    AreaType ENUM('Residential', 'Industrial', 'Business') NOT NULL,
    mobileInfo VARCHAR(100) NOT NULL
);

-- Table: Waste Categories
CREATE TABLE WasteCategories (
    WasteCategoryID INT AUTO_INCREMENT PRIMARY KEY,
    WasteType VARCHAR(50) NOT NULL,
    UnitPricePerKg DECIMAL(10, 2) NOT NULL COMMENT 'Charges per kg
for this type of waste',
    Description VARCHAR(255)
);


-- Table: Collection Schedule (1:1 with Customers)
CREATE TABLE CollectionSchedule (
    ScheduleID INT AUTO_INCREMENT PRIMARY KEY,
    ClientID INT NOT NULL,
    WasteCategoryID INT NOT NULL,
    CollectionFrequency ENUM('Daily', 'Weekly', 'Bi-Weekly',
'Monthly') NOT NULL,
    LastCollectionDate DATE,
    NextCollectionDate DATE,
    FOREIGN KEY (ClientID) REFERENCES Clients_list(ClientID),
    FOREIGN KEY (WasteCategoryID) REFERENCES
WasteCategories(WasteCategoryID),
    CONSTRAINT unique_schedule UNIQUE (ClientID, WasteCategoryID,
CollectionFrequency)
);


-- Table: Waste Disposal (M:N between Customers and Waste
Categories)
CREATE TABLE WasteDisposal (
    DisposalID INT AUTO_INCREMENT PRIMARY KEY,
    ClientID INT NOT NULL,
    WasteCategoryID INT NOT NULL,
    ScheduleID INT NOT NULL,
    DisposalDate DATE NOT NULL,
    QuantityInKg DECIMAL(10, 2) NOT NULL COMMENT 'Amount of waste
disposed',
    FOREIGN KEY (ClientID) REFERENCES Clients_list(ClientID),
    FOREIGN KEY (WasteCategoryID) REFERENCES
WasteCategories(WasteCategoryID),
    FOREIGN KEY (ScheduleID) REFERENCES
CollectionSchedule(ScheduleID),
    CONSTRAINT check_daily_limit CHECK (QuantityInKg <= 5000)
);
```

```sql
-- Table: Billing Records (1:N with Customers)
CREATE TABLE BillingRecords (
    BillingID INT AUTO_INCREMENT PRIMARY KEY,
    ClientID INT NOT NULL,
    BillingDate DATE NOT NULL,
    TotalAmount DECIMAL(10, 2) NOT NULL COMMENT 'Total bill for the
customer',
    FOREIGN KEY (ClientID) REFERENCES Clients_list(ClientID)
);

-- Table: Billing Details (1:N with Billing Records and Waste
Categories)
CREATE TABLE BillingDetails (
    BillingDetailID INT AUTO_INCREMENT PRIMARY KEY,
    BillingID INT NOT NULL,
    WasteCategoryID INT NOT NULL,
    QuantityInKg DECIMAL(10, 2) NOT NULL,
    SubTotal DECIMAL(10, 2) NOT NULL COMMENT 'Calculated as
QuantityInKg * UnitPricePerKg',
    FOREIGN KEY (BillingID) REFERENCES BillingRecords(BillingID),
    FOREIGN KEY (WasteCategoryID) REFERENCES
WasteCategories(WasteCategoryID)
);
```

5. NoSQL: Dockerized Apache Cassandra

5.1 Entity Relation Diagram



5.2 Problem domain and Constraints

A well-designed system that can handle a variety of clients, contract types, and regulatory compliance is necessary for managing customer contracts in the waste management industry. The following issues are addressed by the Cassandra-based schema:

- **Contract Lifecycle Management:** Monitoring contracts in a variety of states, including active, expired, terminated, and broken, and noting the causes of these modifications.
- **Customer Classification:** Supporting the division of customers into groups (premium, general, and discount) according to service requirements and region type.

- **Regulatory Compliance:** Linking Contracts to relevant policies to guarantee compliance with operational and environmental standards.
- **Scalability and Performance:** Facilitating the effective management of large data sets in a dispersed setting and having the capacity to expand horizontally to serve an expanding clientele.
- **Decentralized Relationship Management:** Apache Cassandra relies on manual relationship handling to preserve flexibility and performance, in contrast to relational databases, which impose stringent foreign key constraints.

This schema supports changing business and regulatory requirements while guaranteeing that customer and contract data is highly available and scalable.

5.3 Schemas Explanation and How They Align with the Constraints

The five main tables that make up the schema each have a distinct function in the contract management system:

- **Clients Table:**
  - Records the name, address, phone number, and categorisation (premium, general, or discount) of the client.
  - contains a client_status_id that can be used to connect to the Client Status Table, which monitors compliance or security breaches.
  - For distributed operations over a Cassandra cluster, globally unique IDs are guaranteed by the use of a UUID primary key.
- **Contracts Table:**
  - keeps track of certain customer contract details, such as the type of contract, the base charge, and the expiration dates.
  - keeps track of the contract's lifecycle state (Active, Expired, Terminated, Breached), and for auditing and analytics purposes, it provides a *reason_for_status*.
  - Client_id offers flexibility in connecting client information with contract entries by acting as a manual reference to the Clients Table.
- **Regulatory Policies Table:**
  - outlines waste management rules, each of which is distinguished by a distinct policy_id.
  - Provides details about the policy and its effective date, ensuring alignment with legal requirements.
- **Contract Policies Table:**
  - establishes a many-to-many link between regulatory policies and contracts, enabling contracts to follow several regulations at once.
  - For effective indexing and querying, a composite primary key (*contract_id, policy_id*) is used.

- **Client Status Table:**
  - uses *client_status_id* to identify a client's account and track its current compliance or breach status.
  - keeps track of statuses like "Compliant," "Breach of Policy," or "Suspended," which offer important information for risk and operational management.

This schema's design supports strong contract monitoring, decentralised data management, and regulatory policy compliance, all of which are in line with the issue domain.

5.4 Relationships Between the Tables

The schema describes relationships in a way that strikes a compromise between flexibility and data integrity, even though Cassandra does not impose foreign key constraints:

- **Clients → Contracts:**
  - Contracts and clients have a reasonable one-to-many connection. The *client_id* field in the Contracts Table indicates how many contracts each client may have. Because this relationship is manually maintained, relational database overhead is avoided, and querying is made efficient.
- **Contracts → Contract Policies:**
  - Contracts and regulatory policies are linked in a many-to-many relationship. A contract can abide by several regulations thanks to the Contract Policies Table, which links *contract_id* from Contracts with *policy_id* from Regulatory Policies.
- **Regulatory Policies → Contract Policies:**
  - The Contract Policies Table links contracts to the policies specified in the Regulatory Policies Table, guaranteeing compliance with regulations.
- **Clients → Client Status:**
  - a one-to-one relationship in which the *client_status_id* is used to relate each client in the Clients Table to their account status in the Client Status Table. This makes it possible to track compliance and breach statuses in detail.

Despite being managed by hand, these links provide for a flexible and scalable schema for client contract and compliance management. Even with big datasets and intricate queries, Cassandra's distributed architecture guarantees system performance. This method preserves fault tolerance and scalability while meeting the ever-changing demands of waste management.

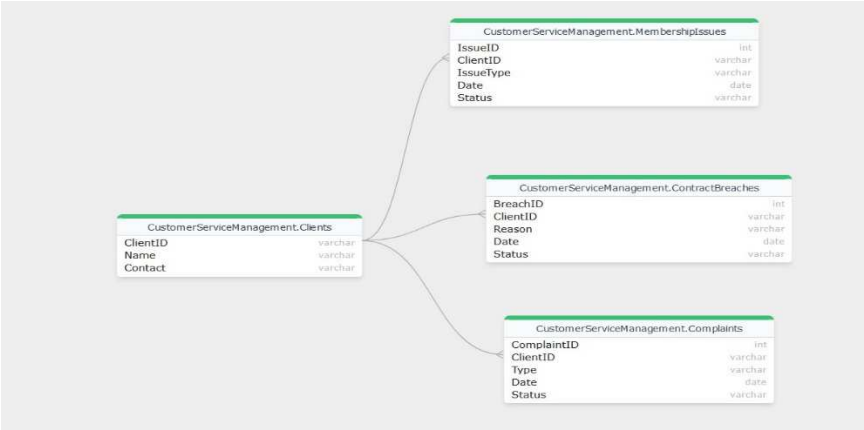
5.5 CQL Code

```
---- Create keyspace
CREATE KEYSPACE IF NOT EXISTS customer_contract_management
WITH replication = {'class': 'SimpleStrategy', 'replication_factor':
'1'};

-- Use the created keyspace
USE customer_contract_management;

-- Create Clients Table
```

```sql
CREATE TABLE clients (
    client_id UUID PRIMARY KEY,
    name TEXT,
    area_type TEXT,            -- Residential, Industrial, Business
    address TEXT,
    contact_number TEXT,
    classification TEXT,       -- Premium, General, Discount
    client_status_id UUID      -- Foreign key reference to
client_status
);

-- Create Contracts Table
CREATE TABLE contracts (
    contract_id UUID PRIMARY KEY,
    client_id UUID,            -- Foreign key reference to clients
(not enforced, manual handling)
    area_type TEXT,            -- Residential, Industrial, Business
    contract_type TEXT,        -- Premium, General, Discount
    start_date DATE,
    end_date DATE,
    status TEXT,               -- Active, Expired, Terminated,
Breached
    base_fee DECIMAL,          -- Base fee associated with the
contract
    last_updated TIMESTAMP,
    reason_for_status TEXT     -- Reason for breach or termination
);

-- Create Regulatory Policies Table
CREATE TABLE regulatory_policies (
    policy_id UUID PRIMARY KEY,
    policy_name TEXT,
    policy_details TEXT,
    effective_date DATE
);

-- Create Contract Policies Table
CREATE TABLE contract_policies (
    contract_id UUID,
    policy_id UUID,
    PRIMARY KEY (contract_id, policy_id)
);

-- Create Client Status Table
CREATE TABLE client_status (
    client_status_id UUID PRIMARY KEY,
    account_status TEXT       -- Enumeration for types of breaches
or compliance issues
);
```

6. XML: Dockerized BaseX

6.1 Entity Relation Diagram



6.2 Problem domain and Constraints

A waste management system's XML database for customer service management is made to effectively manage client-related service records, guaranteeing quick tracking and resolution of problems such as grievances, membership concerns, and contract violations. The following specifications and limitations are met by this database:

- **Customer Support Needs:** keeps track of and arranges membership problems, customer complaints, and contract violations in an orderly manner to guarantee prompt resolution and openness.
- **Scalability:** Because of its adaptability, the XML format can be used for semi-structured data that expands as new customer entries, grievances, or membership difficulties are added.
- **Automation and Bulk Data Generation:** To replicate real-world situations, the system creates sample data programmatically, enabling thorough testing and operation validation.
- **Distributed Access:** The database supports decentralised and effective operations by enabling remote access using a RESTful interface (BaseX).
- **Constraints:**
    - **Dynamic Data Relationships:** Relationships between items must be programmatically controlled because XML does not enforce foreign key constraints by default.
    - **Performance Considerations:** The system must effectively manage many service records (400 clients, 300 complaints, etc.) without getting overloaded.

This solution ensures comprehensive coverage of customer service issues while being flexible enough to adapt to new business requirements.

6.3 Schemas Explanation and How They Align with the Constraints

The key components associated with customer service are captured by the XML representation of the database schema:

- **Clients:**

    - This section stores client details such as *ClientID, Name*, and *Contact*, serving as the foundational entity for linking service records.
    - The segmentation of clients into residential, industrial, or business areas allows tailored service management.
- **Complaints:**
    - Each complaint is identified by a unique *ComplaintID* and linked to a specific client via *ClientID*.
    - Tracks the nature of the complaint (Type), date of filing, and current status (e.g., "Pending," "Resolved").
    - Aligns with the problem domain by categorizing complaints into billing, contract breach, and renewal issues for precise issue handling.
- **Membership Issues:**
    - Captures issues related to membership upgrades or downgrades, identified by IssueID and associated with the client via ClientID.
    - Tracks IssueType, Date, and Status to ensure prompt resolution and smooth service transitions.
- **Contract Breaches:**
    - Stores records of contract violations, such as late payments or policy infractions, with a unique BreachID linked to ClientID.
    - Includes details like Reason, Date, and Status to document breach occurrences and resolutions.

By preserving a hierarchical structure that makes data administration and retrieval easier, the schema complies with the restrictions. Because of XML's flexibility, adding additional fields or elements is simple and won't interfere with the structure already in place.

6.4 Relationships Between the Tables

Although relationships are not inherently enforced by XML, the schema programmatically models logical associations between various elements:

- **Clients → Complaints:**
    - *ClientID* in the Complaints section links back to the relevant client in Clients, indicating a one-to-many linkage between clients and complaints. This makes it possible to effectively trace every complaint made by a particular client.
- **Clients → Membership Issues:**
    - *ClientID* establishes a one-to-many relationship between clients and their membership concerns. Monitoring service upgrades, downgrades, and their statuses is made easier as a result.
- **Clients → Contract Breaches:**

- Clients are linked to contract violations through another one-to-many relationship, which makes it possible to track non-compliance incidents and efforts to resolve them.

Consistency in the ClientID throughout all relevant sections ensures the implicit maintenance of these links. The Python automation code's programmatic handling of these relationships ensures data integrity and makes querying and reporting more effective.

While automation scripts guarantee that the database is consistent, scalable, and in line with the requirements of a dynamic customer service environment, this design makes use of XML's advantages for hierarchical and semi-structured data.

6.5 XML Code:

```
<CustomerServiceManagement>
    <Clients>
        <Client ClientID="" Name="" Contact="" />
    </Clients>
    <Complaints>
        <Complaint ComplaintID="" ClientID="" Type="" Date="" 
Status="" />
    </Complaints>
    <MembershipIssues>
        <Issue IssueID="" ClientID="" IssueType="" Date="" Status="" 
/>
    </MembershipIssues>
    <ContractBreaches>
        <Breach BreachID="" ClientID="" Reason="" Date="" Status="" 
/>
    </ContractBreaches>
</CustomerServiceManagement>
```